

Data Structures and Algorithms Interview Notes

RAG-friendly interview preparation material covering core concepts and explanations.

1. Data Structures

A data structure is a way of organizing data so that operations such as access, insertion, deletion, and searching can be performed efficiently.

2. Time and Space Complexity

Time complexity describes how running time grows with input size. Space complexity describes additional memory usage. Big-O notation is commonly used to express an upper-bound growth rate.

3. Arrays

An array stores elements in indexed positions, usually in contiguous memory. Access by index is fast, while insertion or deletion in the middle may require shifting elements.

4. Strings

A string is a sequence of characters. Common operations include searching, comparison, concatenation, reversal, and frequency counting.

5. Searching

Linear search checks elements one by one and works on unsorted data. Binary search repeatedly divides a sorted search range into two parts.

6. Sorting

Sorting arranges elements in an order. Bubble sort, selection sort, and insertion sort are simple comparison-based algorithms. Merge sort uses divide and conquer, while quicksort partitions around a pivot.

7. Linked Lists

A linked list stores nodes connected using references. A singly linked list has a next reference. A doubly linked list has next and previous references. Linked lists support efficient insertion or deletion when the relevant node position is known.

8. Stack

A stack follows Last In, First Out (LIFO). Common operations are push, pop, and peek. Stacks are useful in recursion, expression evaluation, and undo operations.

9. Queue

A queue follows First In, First Out (FIFO). Common operations are enqueue and dequeue. Queues are used in scheduling and breadth-first search.

10. Hashing

Hashing maps keys to positions using a hash function. Hash tables such as HashMap can provide efficient average-case lookup, insertion, and deletion.

11. Trees

A tree is a hierarchical data structure. A binary tree has at most two children per node. A binary search tree maintains an ordering that supports searching and insertion.

12. Tree Traversal

DFS-style tree traversals include preorder, inorder, and postorder. Level-order traversal visits nodes level by level and is commonly implemented using a queue.

13. Heap

A heap is a complete binary tree satisfying a heap property. In a min-heap the smallest element is at the root; in a max-heap the largest is at the root. Heaps are used to implement priority queues.

14. Graphs

A graph consists of vertices and edges. Graphs can be directed or undirected and weighted or unweighted. BFS explores level by level, while DFS explores deeply before backtracking.

15. Recursion

Recursion is a technique where a function calls itself. A recursive solution needs a base case to stop and a recursive case to reduce the problem.

16. Dynamic Programming

Dynamic programming solves problems with overlapping subproblems and optimal substructure by storing results so that repeated work is avoided.

17. Greedy Algorithms

A greedy algorithm makes a locally best choice at each step with the goal of constructing a global solution. Correctness depends on the specific problem.

18. Two Pointers and Sliding Window

Two pointers maintain two positions in a sequence to reduce unnecessary work. Sliding window maintains a changing contiguous range and is useful for many subarray and substring problems.